
CHAPTER 3

Implementation

In this chapter, we will discuss the technologies used to carry out this work. We will begin by explaining the technologies used for each part, giving the reasons why we have opted for one technology over another. We will explain the technology employed at different levels, such as infrastructure level and functional level of analysis and experiments. Finally, we will describe the framework infrastructure that we have created to enable easy and reproducible dataset creation, experimentation, and analysis of results, all following a systematic, clear, and straightforward process.

3.1 Chosen technologies

Different tools and technologies have been used to carry out this research work. Each one has been used for a specific reason and context, some have been used throughout the whole work and others have been necessary only for a part of it. All of them, independently and together, have formed the working environment.

The work has been mainly undertaken on a personal computer, on which all the necessary software has been installed and configured. Windows has been the main Operating System I have worked on, but the experimental phase was carried out on a virtual machine provided by VRAIN  running on a Linux system.

Python has been the programming language used, and it was chosen because it is currently the most used language in Data Science projects and, therefore, it has many advantages when you use it as there is more documentation and more libraries available. Along with Python, a virtual environment manager for Python, called Anaconda, has been used to create an independent environment for this project with the libraries and their respective versions, without affecting the versions of other environments in other projects.

Anaconda has made it possible, in a simple way, to use the same Python environment on both Windows and Linux. The parts of data analysis and exploration, dataset generation, proof of concept and analysis of results were carried out on Windows. For these tasks we used Jupyter Notebooks integrated with Visual Studio Code. Jupyter Notebooks is an application that allows you to implement and run code by execution cells, so it is very useful for analytical tasks or code proofs, and it also permits to take notes in Markdown and Latex language.

However, Notebooks are not the most suitable tool for running experiments, so the neces-

sary code was implemented in a self-made Python library to have an automated pipeline in which the data is preprocessed, the training is executed and the results are generated. This pipeline is executed by receiving as input the configuration and parameters of the experiment. At the end, the different results of each configuration are generated, as well as running logs to verify that it has worked correctly and to analyse its performance. Finally an Excel file is generated to summarise the whole experiment. All this pipeline is also made to be executed in the Linux BASH terminal, passing the configuration through parameters in the terminal call or through a JSON configuration file. We have also used TMUX, a multiplexer for UNIX systems that allows a terminal to be divided into multiple independent sessions and thus be able to run several experiments in parallel on a single terminal.

Some of the Python libraries that have been used throughout the work have been NumPy, an open source library that supports the creation of vectors and multidimensional arrays as well as providing a large collection of high-level mathematical functions to operate with them; and Pandas, a library specialised in the manipulation and transformation of data in the form of tables, called Dataframes. Both libraries are generic libraries that are widely used for data analysis and manipulation, and therefore have a large documentation and support community behind them. They have been used both in the database creation stage, cleaning and transforming the original data and creating new features, as well as for data analysis and exploration and data processing before and during model training. In addition, Pandas has been used for more in-depth analysis of the results.

On the other hand, there are libraries that have been used for more specific functions. This is the case of the graphic generation libraries, Matplotlib and Plotly. These two libraries have been used to visually analyse data from the matches as well as data about the performance of the prediction models, such as the learning curve. Matplotlib has been used more for data analysis, as well as for creating the graphical content shown in this document. On the other hand, Plotly, which provides interactive graphics, has been used more for the analysis of the data in a more interactive and comfortable way.

The main Machine Learning library that has been used is Scikit-Learn, which provides a wide palette of functions for data preprocessing and scaling, which have been used during data preparation before training or for feature selection, such as with ANOVA or Variance Threshold. This library has also been used for Baseline models, as Scikit-Learn offers an API for classical ML algorithms such as SVM or Decision Trees. We have also used Scipy, a library that offers a collection of mathematical functions and algorithms similar to Matlab, which has been used mainly during the data analysis phase.

Two libraries have been used to implement the predictive models, each one specialised in a different Machine Learning algorithm. On the one hand, PyTorch has been used for the ANN-based models. This library is one of the two most widely used libraries for ANN, and it was decided to use PyTorch rather than the other, Tensorflow, because we already had some academic experience with PyTorch. On the other hand, for predictive models based on Gradient Boosting models, the LightGBM library has been used, which implements a super-efficient version of this algorithm and has recently outperformed other versions.

Finally, for the optimisation of the hyperparameters of the ANN and GB models, we have used Optuna, a framework that automatically performs a Bayesian search for hyperparameters by simply defining and passing an objective function to be minimised or maximised.

In order to be able to carry out all this work, not only did we need the appropriate technology to carry it out, but also an infrastructure and a software platform on which to conduct the entire workflow. In the next section we will talk about how this infrastructure has been implemented so as to ease the work undertaken.

3.2 Platform development

In this section we will describe the software platform on which we have carried out all our work. Each part of the total has required specific tools and infrastructure, so in many cases this infrastructure has been implemented as we have reached that part.

First of all, as we have already mentioned in the previous section, the data analysis part has been carried out entirely in Jupyter Notebooks, because it is more suitable for this kind of exploratory tasks that require a lot of creativity to perform, and the fact of running the code in parts (cells) makes it much easier.

The generation of the datasets has also been done with Notebooks. This is due to two main reasons. On the one hand, Notebooks are a really convenient tool for testing or developing code by monitoring the state of execution as it is being executed and developed at the same time. In the case of dataset generation, in which many transformations and filters are applied to the data, it is very interesting to see at which point the execution is in order to validate the code. The second reason is that the generation of the dataset only takes place once, since the dataset is invariant, except if we want to modify some characteristic, for which we would go to the Notebook. If we wished to add data to the dataset periodically in an automated way, we would have been interested in moving the code to other more convenient code formats.

Finally, the experiments have been the most complex part to implement. Unlike the dataset that is run once and that's it, the experiments are a part that you want to run multiple times, with different configurations and maintaining reproducibility when running the same configuration. Reproducibility in Data Science experiments implies that the results obtained must be able to be replicated using the same data, code and parameters at any time. To ensure this reproducibility, a seed is used, which is an initial value used in the generation of pseudo-random numbers to guarantee that the results are reproducible. By setting a seed, the same sequence of pseudorandom numbers can be obtained in different executions of the code.

In addition, the experiments, as well as receiving an input, which would be the configuration and data, return an output, which are the results and logs of the execution. These experiments must be able to be executed on both local and remote machines, even if they have different operating systems.

To set up the compatible infrastructure on both systems, the anaconda virtual environment has been cloned from the local machine to the remote one, and the paths from which files are read and written have been translated on both systems, dynamically selecting them using a system variable that indicates their type of system. On the other hand, the file transfer protocols SCP (Secure Copy) and SSH (Secure Shell) have been used to send files and launch executions on the remote machine.

When carrying out an experiment, we do not want to waste a lot of time indicating its

parameters to the execution, nor later generating and searching for its results. That is why this whole process has been implemented to be simple, monotonous, error-avoiding and efficient.

The parameters of the experiment are indicated through a `YAML` file where there are several fields: Data, Model and Results. In the Data field, you indicate which data you want to train the model with, how many features you want the model to train - indicating which features you want to keep, which ones you want to delete and which dimensionality reduction technique you want to apply - and other parameters such as the scaling method to apply to the data. In the Model section, you indicate which algorithm you want to run with, which type of optimisation you want to train the hyperparameters (Random search or Bayesian search). It is also possible to manually set hyperparameters to specific values in case you want to analyse a specific configuration. In the Results section we can indicate if we want to save the model predictions, as well as the logs and metrics. We also indicate the path in which we want it to save as well as the possibility of indicating some metrics that we want it to calculate, such as the F1-score or to group different runs and calculate grouped statistics (such as the performance of a hyperparameter in different experiments).

In order to execute the experiments, two libraries have been created, one for each algorithm, ANN and LightGBM, which are invoked from another file that is being executed with the Bash terminal. In the ANN library, functions have been declared to train and validate a model, to train with Cross-Validation, the function that performs the different experiments of a hyperparameter optimisation process (RS or GS) has been implemented and logging functions have also been implemented, both some using native PyTorch functions such as hooks to log the performance of a model (gradients and activation functions) and others to log the error and the error in each epoch. In the LightGBM library, Bayesian search training has been implemented with Optuna by defining a function to optimise (minimise error) given a model metric.

All these results and logs are written in a single, equally structured directory for each experiment, allowing multiple results to be read in the same way. In addition, after the execution of a set of experiments, a script is run in which an Excel is created summarising all the experiments done and giving an overview of how the experiments went.

These results are then transferred to the local machine where, with a Jupyter Notebook, they are analysed with the generation of dashboards or the exhaustive analysis of predictions.

